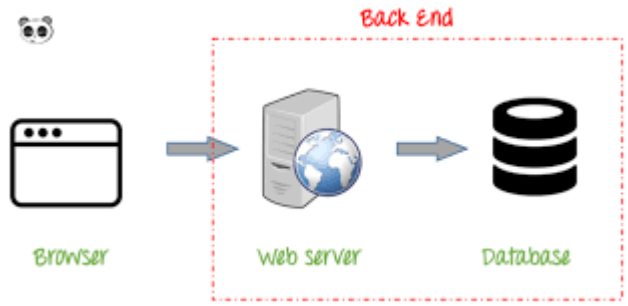
The back-end is the "hidden" part of a website, where logic, databases, and features that users don't directly see in their browser are handled.

- **Main functions**Receive requests from the front-end, process business logic, interact with the database, and send responses back to the client.
- **Core components**This includes the Server, Application, and Database.




The back end includes the following core components:

- **Server:**This is where services run, processing and executing logic, receiving/sending requests and responses via HTTP(S), WebSocket, gRPC, etc.
- **Database:**A place to store, retrieve, manage, and maintain data consistency. This includes both relational database management systems (SQL: MySQL, PostgreSQL, Oracle) and non-relational systems (NoSQL: MongoDB, Redis, Cassandra).
- **API (Application Programming Interface):**This intermediary bridge allows Front End or external systems to access resources and execute functions on the server in a standardized and secure manner.
- **Background services:**Perform asynchronous tasks such as sending emails, image processing, heavy computation, cron jobs (scheduling tasks), and message queues.
- **Security layers:**This includes authentication, authorization, encryption, access control, logging, auditing, and intelligence-based detection and response systems (IDS/IPS).

CKey programming languages that help build robust server architectures:

- **PHP**The most popular language for the web, easy to learn, and with a large support community.
- **Python**Popular due to its clear syntax, often bundled with frameworks like Django or Flask.
- **Node.js**Allows the use of JavaScript on the server side, helping to synchronize the language between the front-end and back-end.

- **Java**Often used in large-scale (enterprise) projects due to its high security and stability.
- **Ruby**Famous for its Ruby on Rails framework, which helps in rapid application development.

This is where all the website's data is stored (user information, articles, products, etc.).

- **Relational database (SQL)**Use tables to manage structured data. Examples include MySQL, PostgreSQL, and Microsoft SQL Server.
- **Non-relational databases (NoSQL)**Suitable for flexible data, not rigidly formatted. Examples: MongoDB, Redis.

A typical back-end workflow unfolds as follows:

1. **Reception**Listen for requests sent from the user's browser via the HTTP/HTTPS protocol.
2. **Handle**: Checks data validity, authenticates users, and performs logical calculations.
3. **Access**Connect to the database to read, write, or update information as needed.
4. **Feedback**Package the processing results (usually in JSON or HTML format) and send the response back to the front-end for display.

The back end largely determines the quality, security, and scalability of a software product:

**Core business process handling:**

- Implementing complex rules and algorithms, and performing dynamic calculations.
- Develop business workflows for finance, commerce, healthcare, education, industry, AI, and IoT sectors.

**Data management:**

- Optimal database design: relational modeling, data normalization, index optimization, partitioning, replication, sharding.
- Ensure ACID (Atomicity, Consistency, Isolation, Durability) for the trading system.
- Big Data processing: streaming, batch processing, caching.

**Security and access control:**

- Multi-factor authentication (2FA, MFA), JWT/OAuth2 token authentication.
- User permission management by role (RBAC, ABAC).
- Implement encryption measures (at rest, in transit), protect personal information (PII), and comply with GDPR and HIPAA standards.

- Implement logging and monitoring of contact tracing (audit trail).

**Performance and availability:**

- Load balancing, horizontal scaling, vertical scaling.
- Use cache (Redis, Memcached), CDN, tối ưu API response time, batching request, rate limiting.
- Failover, backup, auto-recovery, and automatic system monitoring mechanisms (Prometheus, Grafana).

**Cross-platform integration:**

- Connect internal and external APIs (RESTful, GraphQL, gRPC, WebSocket).
- Integrates payment systems (Stripe, PayPal), email, SMS, and push notifications.
- Communicate with microservices and message brokers (RabbitMQ, Kafka).

**DevOps and CI/CD support:**

- It plays a central role in the automated build, test, and deployment process.
- Supports containerization (Docker, Kubernetes), cloud deployment (AWS, Azure, GCP).
- Đảm bảo versioning API, backward compatibility, auto-scaling.

**Serving modern software models:**

- Microservices: breaking down modules into smaller parts, increasing independent scalability.
- Event-driven: asynchronous, real-time processing.
- Serverless: Reduces operating costs, automatically scales with demand.

**Important supporting concepts**

- **API (Application Programming Interface)**It acts as a "bridge" allowing the front-end and back-end to communicate with each other.
- **Authentication & Authorization**Manages user logins and access permissions.
- **Server Management**Managing server infrastructure, which can be physical servers or cloud services such as AWS, Google Cloud, and Azure.

## Database

DatabaseIt is the central location for storing, organizing, querying, and protecting the system's data. The choice depends on the type of data, scalability, and consistency requirements.

**Relational Database Management System (RDBMS):**

- **MySQL:**Open-source, stable performance, master-slave replication, good integration with PHP and Node.js.
- **PostgreSQL:**Strong support for complex queries, unstructured data (JSONB), high scalability, and strict ACID compliance.
- **MS SQL Server, Oracle:**Optimized for large enterprises, with numerous security features, backup, auditing, and detailed access control.

**NoSQL databases:**

- **MongoDB:** Document-based, JSON-like, schema flexible, horizontal scaling, aggregation pipeline mạnh mẽ.
- **Redis:** In-memory key-value store, tối ưu caching, pub/sub, session management.
- **Cassandra:** Distributed, phù hợp big data, writes throughput cao, replication multi-data center.
- **Couchbase:** Hybrid JSON document + key-value, indexing mạnh, scaling tốt.

**Quick comparison (list):**

- RDBMS: ACID, strict schema, robust in transactions, suitable for financial systems.
- NoSQL: Base-based, flexible schema, optimized for horizontal scaling, suitable for social networks, IoT, and Big Data.

**Technical factors to consider:**

- Loại dữ liệu (transactional/analytical, structured/unstructured)
- High query response capacity, low latency.
- Availability (HA), backup, failover
- Integration capabilities with the DevOps and Cloud ecosystems.

## APIs and Web Services

APIWeb services are standardized communication interfaces between the back end and the front end, mobile app, or external systems, ensuring separation between the display layer and the business logic layer.

- **RESTful API:**
  - Chuẩn HTTP method (GET, POST, PUT, DELETE), resource-based URL, trả về JSON/XML.
  - It supports caching, versioning, and rate-limiting, and is popular due to its simplicity.
- **GraphQL:**

    - Single endpoint, client-defined queries, optimized payload (avoiding underfetch/overfetch).
    - Real-time support via subscription is effective when data is linked across multiple entities.
- **gRPC:**
    - RPC protocol sử dụng HTTP/2, Protocol Buffers, high-performance, low-latency.
    - Optimized microservices system, suitable for parallel processing and internal communication.
- **SOAP:**
    - XML-based standard, supporting WS-Security and transactional data, ensuring data integrity and security, commonly found in banking and insurance systems.

**API technical requirements:**

- Authentication/Authorization (JWT, OAuth2, API Key)
- Throttling, rate limit, request validation, input sanitization
- Quản lý versioning (URI, header, custom media type)
- Documentation chuẩn hóa (OpenAPI/Swagger, GraphQL Playground)

### Related DevOps tools

DevOps tools enhance automation, standardize the development-testing-deployment process, ensure continuous back-end operation, and facilitate easy scalability, rollback, and monitoring.

- **Source code management:**
    - Git (GitHub, GitLab, Bitbucket): Versioning, branching, code review, CI/CD integration.
- **CI/CD:**
    - Jenkins: Pipeline scripting, integrated automated testing, building, and deployment.
    - GitHub Actions/GitLab CI: Workflow-as-code, easy integration, multi-environment deployment.
- **Containerization and orchestration:**
    - Docker: Packaging applications, easily migrating environments. Research by Burns and Beda (2019) in "Kubernetes: Up and Running" from Google demonstrates that Kubernetes orchestration reduces operating costs by 70% and increases resource efficiency by 300%. According to the CNCF survey (2022), organizations using Kubernetes report a 50% reduction in deployment cycles and a 60% reduction in

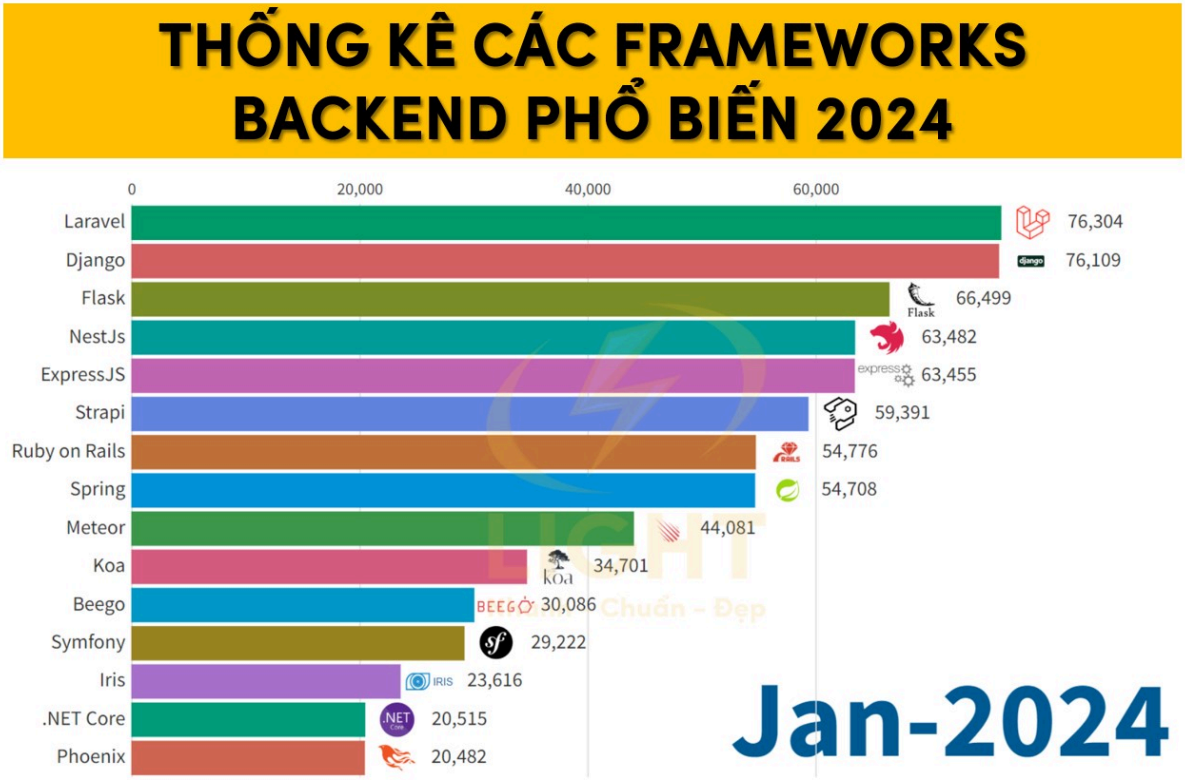
infrastructure costs. Research by Red Hat (2021) shows that applications containerized with Kubernetes have 99.95% availability and support auto-scaling up to 5000 nodes with response times under one second for pod scheduling.
    - Kubernetes: Orchestration, autoscaling, zero-downtime deployment, service discovery.
    - Terraform, Ansible: Infrastructure as Code, configuration management, resource provision automation.
- **Monitoring & Logging:**
    - Prometheus/Grafana: Metrics collection, alerting, visualization.
    - ELK Stack (Elasticsearch, Logstash, Kibana): Focuses on logging, analytics, and dashboards.
    - Sentry: Error tracking, alerts, classification, and monitoring of production errors.

**Standard DevOps process:**

1. Code: Version management with Git.
2. Build: Automate artifact packaging (Docker, Maven, npm).
3. Test: Unit testing, integration testing, code analysis.
4. Deploy: Continuous deployment to the staging/production environment.
5. Monitor: Track health checks, performance, logs, and incident alerts.

**DevOps performance metrics:**

- Deployment frequency
- Malfunction Recovery Time (MTTR)
- Deployment success rate
- Pipeline automation level




**Criteria for evaluating frameworks:**

- Execution performance (benchmark requests/sec)
- Scalability (scale-out/scale-in)
- Security features (XSS, CSRF, injection prevention)
- ORM management, database migration
- Support for API development (RESTful, GraphQL)